

Iris 光影开发笔记 (1)

目录

1 第 0 章 计算机图形学基础	2
1.1 CPU 与 GPU 的架构分野	2
1.2 OpenGL 与 GLSL	2
1.3 渲染管线的数学骨架	3
1.4 Iris 渲染管线全景	3
2 第 1 章 GLSL 与 Base-330 初探	5
2.1 类型系统与向量代数	5
2.2 in/out 接口与顶点插值	5
2.3 #version 330 compatibility 的工程权衡	5
2.4 实战：世界反相与光照顺序的非交换性	6
3 第 2 章 G-Buffers 阶段详解	7
3.1 gbuffers 与 composite 的分界	7
3.2 Minecraft 光照模型的代数结构	7
3.3 RENDERTARGETS：从单输出到多输出	7
3.4 延迟渲染的收益与代价	8
4 第 3 章 Deferred 延迟光照	9
4.1 坐标空间管线与逆向重建	9
4.2 Phong 光照模型	10
4.3 Blinn-Phong 改进	11
4.4 deferred.fsh 的完整实现范式	11
4.5 延迟渲染的性能考量	12
5 第 4 章 Shadow Mapping 阴影	13
5.1 核心思想	13
5.2 数学推导	13
5.3 阴影图的空间分辨率	14
5.4 Shadow Acne：量化误差的机理与修正	14
5.5 Peter-Panning 效应	15
5.6 PCF 软阴影	15
5.7 最小可用实现	16

1 第 0 章 计算机图形学基础

1.1 CPU 与 GPU 的架构分野

实时渲染的根本约束是吞吐量 (Throughput)，而非延迟。以 1920×1080 分辨率为例，单帧像素数

$$N_{\text{px}} = 1920 \times 1080 \approx 2.07 \times 10^6$$

若每像素需要 k 次浮点运算，单核 CPU 以时钟频率 f 、每周期 p 条浮点指令运行，完成一帧所需时间为

$$t = \frac{N_{\text{px}} \cdot k}{f \cdot p}$$

取 $f = 5 \text{ GHz}$, $p = 4$, $k = 50$: $t \approx 5.2 \text{ ms}$; 若 $k = 200$ (更接近真实光照复杂度), $t \approx 20.7 \text{ ms}$, 帧率上限约 48 FPS, 且此过程中核心完全被占用, 无法处理游戏逻辑。**CPU 单线程吞吐量不足是 GPU 存在的根本原因。**

GPU 采用 **SIMT (Single Instruction, Multiple Threads)** 架构: 32 个 (NVIDIA Warp) 或 64 个 (AMD Wavefront) 线程构成一个调度单元, 同一时刻执行同一条指令、各自操作不同数据。这决定了两条重要约束:

- **分支串行化:** Warp 内 `if/else` 的两条分支依次执行、以掩码方式屏蔽无效线程, 最坏情况下开销加倍;
- **合并访存 (Coalesced Access):** 同一 Warp 访问连续显存地址时可合并为一次内存事务, 随机访存则显著降速。

表 1: CPU 与 GPU 架构对比

特性	CPU	GPU
核心数	4 ~ 32	数千至数万
调度粒度	硬件线程 1 ~ 2/核心	Warp (32 线程) /调度器
分支处理	分支预测 + 乱序执行	Warp 内串行化
内存模型	大缓存、低延迟	高带宽、高延迟 (需以并行度隐藏)
适用场景	逻辑、物理、AI	顶点变换、像素着色、矩阵运算

关键推论: GLSL 程序的语义是”为每个顶点/像素各自独立执行一次 `main()`”, 而非一次性作用于全部数据。着色器内不存在跨线程通信 (无共享内存语义), 也不能依赖执行顺序——这是后文”延迟渲染要把光照集中到全屏 Pass”这一设计动机的底层原因: 全屏 Pass 的像素数固定, 与场景几何复杂度解耦。

1.2 OpenGL 与 GLSL

OpenGL 是 Khronos Group 维护的规范而非实现, 定义函数签名、状态机语义与 GLSL 语法, 具体翻译为硬件微码由厂商驱动完成。它以**状态机**方式工作: `glUseProgram()`、`glBindTexture()` 等调用修改全局状态, 后续绘制指令按当前状态执行; Minecraft Java 版通过 LWJGL 绑定 OpenGL, Iris 负责拦截绘制调用、切换着色器程序、设置 uniform。

表 2: GLSL 版本演进

版本	关键变化	意义
GLSL 1.20	varying / attribute	早期光影基线
GLSL 3.30	in / out、layout	本教程编译目标 (Base-330)
GLSL 4.60	计算着色器、layout(binding=...)	Base-460 高级基线

着色器程序的生命周期：编译（.vsh/.fsh 源码解析，失败则 `glGetShaderInfoLog()` 写入日志）→ 链接（校验 in/out 接口匹配）→ Uniform 定位（Iris 自动完成）→ 绘制调用 → GPU 执行（顶点着色器 → 图元装配 → 光栅化 → 片段着色器 → 逐片元测试 → 写帧缓冲）→ 交换缓冲。文件缺失或编译失败时 Iris 不会崩溃，而是回退到内置默认着色器，这是光影包可以”只提供要修改的文件”的工程基础。

1.3 渲染管线的数学骨架

一个顶点从模型空间到屏幕像素，要经过五次变换，其复合形式贯穿全书：

$$\vec{v}_{\text{clip}} = P_{\text{proj}} \cdot V_{\text{view}} \cdot M_{\text{model}} \cdot \vec{v}_{\text{model}}$$

表 3: 坐标变换阶段

阶段	数学操作	输出空间
模型变换	$M\vec{v}$ （旋转 + 平移 + 缩放）	世界空间
视图变换	$V\vec{v}$ （Look-At 构造）	视图空间
投影变换	$P\vec{v}$	裁剪空间
透视除法	$\vec{v}/w$ （硬件自动）	NDC $[-1, 1]^3$
视口映射	$\frac{x_{\text{ndc}} + 1}{2} \times w_{\text{screen}}$	屏幕像素

贯穿全书的洞察：固定管线函数 `ftransform()` 等价于 $P \cdot V \cdot M$ 的合并计算；第 4 章的 Shadow Pass 之所以”从光源视角看场景”，只是把 V_{view} 换成 V_{shadow} ——几何结构与数学形式完全不变，变的只是参照系。这一等价性是理解本笔记后续所有内容的公理。

1.4 Iris 渲染管线全景

Setup/Begin → Shadow Map → Shadow Composite → Prepare
→ G-Buffers → Deferred → Translucent → Composite 1..n → Final

表 4: Iris 渲染管线各阶段

阶段	输入	输出	GPU 工作类型
Shadow	世界几何	shadowtex0/1	顶点 + 片段（仅深度）
G-Buffers	世界几何、纹理	colortex0-7	顶点 + 片段（材质属性，逐真实三角形）
Deferred	colortex*、depthtex*	colortex0-7	片段（全屏四边形）
Composite	colortex*、depthtex*	colortex0-7	片段（全屏四边形）
Final	colortex0	屏幕	片段（全屏四边形）

核心区分：G-Buffers 处理的是真实几何，每个三角形都要走一次顶点着色器；Deferred/Composite/Final 处理的是覆盖全屏的两个三角形，片段着色器对每个像素独立执行一次，与场景复杂度无关。因此复杂计算应尽量后置到 **Composite** 阶段。

2 第 1 章 GLSL 与 Base-330 初探

2.1 类型系统与向量代数

GLSL 是面向 GPU 并行执行设计的类 C 语言。核心数据全部以向量形式流动：**vec2**（UV/屏幕坐标）、**vec3**（位置/方向/RGB）、**vec4**（齐次坐标/RGBA）。分量既可用 **.xyzw**（几何语义）访问，也可用 **.rgba**（颜色语义）访问——两者是同一内存的不同命名视图，这一等价性源自颜色空间本身就是三维（或四维）向量空间：红/绿/蓝是它的基向量，任意颜色是其线性组合，因此 **color1 * color2** 的逐通道相乘对应物理上“两个滤波器串联”。

点积是向量代数中使用频率最高的运算：

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos \theta$$

当 $\|\vec{b}\| = 1$ 时，点积即为 $\vec{a}$ 在 $\vec{b}$ 方向上的投影长度。这一事实解释了渲染代码中反复出现的 **max(dot(N, L), 0.0)**：它是入射光方向在法线方向上的投影，即 Lambert 漫反射强度的来源（详见 §3.3）。

矩阵乘向量的几何直觉同样建立在线性组合之上：

$$M\vec{v} = v_1 \text{Col}_1(M) + v_2 \text{Col}_2(M) + v_3 \text{Col}_3(M) + v_4 \text{Col}_4(M)$$

即矩阵的每一列，就是对应基向量变换后的去向：视图矩阵的列是相机的右/上/前基向量，旋转矩阵的列是旋转后的坐标轴。GLSL 采用列主序存储，但绝大多数场合只需用 ***** 运算符，无需关心内存布局。

2.2 in/out 接口与顶点插值

顶点着色器与片段着色器之间通过同名的 **out/in** 变量传递数据：

Listing 1: 顶点-片段接口示例

```
1 // .vsh
2 out vec2 texcoord;
3 void main() { texcoord = (gl_TextureMatrix[0] * gl_MultiTexCoord0).xy; }
4
5 // .fsh
6 in vec2 texcoord;    // GPU 按重心坐标 (Barycentric) 自动插值
```

顶点着色器仅在顶点上执行；光栅化阶段 GPU 按三角形内部的重心坐标对顶点属性做线性插值；片段着色器在像素上执行，拿到的已是插值结果。**uniform** 变量则是 CPU 侧传入的全局常量，同一次绘制调用内所有线程读到的值完全相同，由 Iris 自动填充，无需手动赋值。

2.3 #version 330 compatibility 的工程权衡

compatibility 关键字保留了固定功能管线时代的内置变量：**gl_Vertex**（模型空间顶点位置）、**gl_Normal**、**gl_Color**、**gl_MultiTexCoord0/1**（纹理坐标/光照贴图坐标）、**gl_ModelViewMatrix**、**gl_ProjectionMatrix**，以及等价于 $P \cdot V \cdot M \cdot \vec{v}_{\text{model}}$ 的函数 **ftransform()**。保留它们的原因是 Minecraft 引擎本身仍以固定管线的数据格式向 GPU 提交顶点——若改用 **core** 模式，则需手动声明顶点属性、绑定 VBO、管理矩阵，这对光影包开发是不必要的复杂度。代价（驱动需同时支持两套管线、macOS 等平台支持受限）在 Minecraft Java 版语境下可忽略。升级到 **#version 460 core**（Base-460）的价值主要在计算着色器（预计算大气 LUT、流体模拟）与显式绑定点，属于进阶话题。

2.4 实战：世界反相与光照顺序的非交换性

修改 `gbuffers_terrain.fsh`:

```
1 color = vec4(1.0 - albedo.rgb, albedo.a);
```

看似简单的取反，暴露了一个贯穿光影调参的代数事实：乘法可交换，但取反不是线性运算，因此操作顺序改变结果。设纹理颜色为 C_{tex} ，光照为 L ：

$$\text{先反相再乘光: } (1 - C_{tex})L = L - C_{tex}L \quad \text{先乘光再反相: } 1 - C_{tex}L$$

两者仅在 $C_{tex}L = \frac{1}{2}$ 时相等。这不是特例，而是所有光影包后处理调参的本质：色调映射、伽马校正、雾效混合，插入的先后顺序本身就是效果的一部分。

3 第 2 章 G-Buffers 阶段详解

3.1 gbuffers 与 composite 的分界

gbuffers_* 系列文件处理真实几何（地形、实体、水、天气等 16 类 Pass），每个三角形都触发一次顶点着色器；composite/deferred/final 处理的是覆盖全屏的两个三角形，片段着色器对每个屏幕像素独立执行，与场景复杂度无关。这一分界是“复杂特效放在 composite 阶段”这一经验法则的根源。

3.2 Minecraft 光照模型的代数结构

原版 Minecraft 的着色模型是三个标量场的逐通道相乘：

$$\vec{C}_{\text{final}} = \vec{C}_{\text{tex}} \cdot c_{\text{vertex}} \cdot \vec{C}_{\text{lightmap}}(u, v)$$

其中不含法线（不区分朝向）、镜面高光、AO、阴影——这正是原版画面“扁平”的数学原因。光照贴图 $L : [0, 1]^2 \rightarrow \mathbb{R}^3$ 编码“方块光 $\times$ 天空光”的联合分布，双线性纹理采样自动实现四邻域加权平均：

$$L(u, v) \approx \sum_{i,j} w_{ij} L(u_i, v_j)$$

这解释了火把亮度过渡为何是平滑渐变而非阶梯。光影包在此基础上叠加：法线贴图漫反射、Shadow Mapping、SSAO、GI 等增强项，各自独立地作为额外因子或加项接入。

3.3 RENDERTARGETS：从单输出到多输出

单输出写法：

```
1 /* RENDERTARGETS: 0 */
2 layout(location = 0) out vec4 color;
```

延迟渲染需要同时写多个附件，把材质属性拆分存储，而不是像前向渲染那样直接算出最终颜色：

Listing 2: 多目标渲染示例

```
1 /* RENDERTARGETS: 0,1,2 */
2 layout(location = 0) out vec4 albedo;          // -> colortex0: 反照率
3 layout(location = 1) out vec4 normalData;      // -> colortex1: 法线(压缩)+深度
4 layout(location = 2) out vec4 material;        // -> colortex2: 粗糙度/金属度/AO
5
6 void main() {
7     albedo = texture(gtexture, texcoord) * glcolor;
8     normalData.xyz = normal * 0.5 + 0.5;      // [-1,1] -> [0,1] 编码
9     normalData.w = gl_FragCoord.z;
10    material = vec4(0.5, 0.0, 0.0, 1.0);
11 }
```

通道打包是显存带宽优化的常见手法：一张 vec4 贴图若只存三分量法线，alpha 通道便被浪费，因此常把深度或其他标量塞进第四通道，换取后续阶段少一次纹理采样。这也是为什么读到 `uniform sampler2D colortex1; // normal + depth` 这类注释时，不能想当然认为它一定在当前文件里被

用满——教程代码常在多章节间复用 uniform 声明模板，某个特定 Pass 可能只用到其中一部分。默认精度为 RGBA8（每通道 8 位），法线精度约 0.004，多数场景可接受；对精度敏感的数据可在 `shaders.properties` 中声明为 RGBA16F/RGBA32F，代价是牺牲旧版 Iris/OptiFine 兼容性。

3.4 延迟渲染的收益与代价

表 5: 延迟渲染的权衡

优势	代价
光照复杂度与几何量解耦	无法直接处理透明物体（需单独前向渲染）
支持大量光源	高显存带宽（多重视口读写）
天然支持 SSAO/SSR	不支持 MSAA（需 FXAA/TAA 替代）
光照计算统一在全屏 Pass	G-Buffer 填充本身有开销

4 第3章 Deferred 延迟光照

4.1 坐标空间管线与逆向重建

延迟光照阶段拿到的只有屏幕像素与深度，必须从这两者**逆向重建**三维位置。这要求先把正向管线的六个坐标空间理清：

$$\text{模型空间} \xrightarrow{M} \text{世界空间} \xrightarrow{V} \text{视图空间} \xrightarrow{P} \text{裁剪空间} \xrightarrow{\div w} \text{NDC} \xrightarrow{\text{视口映射}} \text{屏幕空间}$$

表 6: 坐标空间对照

空间	参照系	典型用途
模型空间	物体自身中心	顶点原始数据，如 <code>gl_Vertex</code>
世界空间	场景统一原点	光源变换的中转站（正向管线在此分叉出 V_{shadow} ）
视图空间	摄像机为原点	光照计算的天然坐标系： <code>viewDir = normalize(-viewPos)</code>
裁剪空间	投影后、未除 w	GPU 据此裁剪视锥体外的图元
NDC	$[-1, 1]^3$ 标准立方体	与分辨率、宽高比无关
屏幕空间	像素坐标	即 <code>gl_FragCoord.xy</code>

每一步在解决什么问题——把上表的抽象定义还原成工程动机：

- **模型 → 世界**：一个模型自身的顶点坐标以模型中心为原点（比如一个方块的顶点坐标分布在 $[-0.5, 0.5]^3$ ）。世界空间把所有物体统一放进“整个场景”的坐标系，唯有在这里，“太阳在哪、玩家在哪、两个物体的相对位置”才具备明确的度量意义。
- **世界 → 视图**：视图变换把坐标原点重新搬到摄像机上，使摄像机永远“位于原点、面朝 $-Z$ 方向”。这一步的价值在于：后续的光照计算（如 `viewDir`）可以直接用“是否指向原点”判断朝向摄像机的方向，无需关心摄像机在世界里的具体位置。
- **视图 → 裁剪**：投影矩阵把视锥体（摄像机可见的四棱锥/长方体范围）压缩为一个规整的立方体，并让离摄像机越远的物体在 x/y 方向上“看起来越小”——但此时还未做除法，故称“裁剪”空间：GPU 依据这一坐标判断哪些图元落在视锥体外、应被裁剪。
- **裁剪 → NDC**：做 $\vec{v}/w$ 的透视除法，这才是“近大远小”真正生效的一步（ w 通常等于原本的深度值，越远 w 越大，除完后 x/y 被压缩得越厉害）。此后所有可见物体都被规整进 $[-1, 1]^3$ 的标准立方体，与分辨率、宽高比无关。
- **NDC → 屏幕**：把 $[-1, 1]$ 映射到实际像素坐标 $[0, \text{viewWidth}] \times [0, \text{viewHeight}]$ ，此后即为 `gl_FragCoord.xy`。

关键点：方向是可逆的。上述五步是**正向管线**——顶点着色器渲染时走的路径。而 `screenToView/screenToWorld` 这类函数做的是**反向操作**：拿到屏幕像素的位置与深度，沿链条往回“倒推”到视图空间或世界空间，用的是**逆矩阵**（...Inverse），原理与正向完全对称，只是方向相反：

- **screenToView**：屏幕 UV + 深度 $\rightarrow$ NDC $\rightarrow$ 左乘 `gbufferProjectionInverse` $\rightarrow$ 视图空间（对应链条中“裁剪/NDC $\rightarrow$ 视图”这一段反着走）；
- **screenToWorld**：在此基础上再左乘一次 `gbufferModelViewInverse`，多退一步到世界空间；

- `shadowModelView`、`shadowProjection`：则是另一条完全独立的正向管线——只是把“摄像机”换成了太阳/月亮，世界空间中的点重新走一遍 $\text{View} \rightarrow \text{Projection} \rightarrow \text{NDC}$ 的正向流程，落到阴影贴图自己的像素坐标系里（详见第 4 章）。

一句话总结：模型/世界/视图/裁剪/NDC/屏幕，本质都是同一个点，只是站在不同参照系下观察它的坐标值；矩阵负责把点从一个参照系搬到下一个，逆矩阵负责搬回去。光影编程里绝大多数“变换类”代码，都是在这条链条上来回搬运坐标——记熟这张图，看到任何 `xxxToYyy` 函数名都能立刻反应出它在链条的哪一段、往哪个方向走。

逆向重建对应上述链条反向应用逆矩阵：

$$\vec{v}_{\text{view}} = (P^{-1} \vec{v}_{\text{ndc, homog}}) / w, \quad \vec{v}_{\text{ndc, homog}} = (2uv - 1, 2d - 1, 1)$$

对应 GLSL：

Listing 3: `screenToView` 实现

```
1 vec3 screenToView(vec2 uv, float depth) {
2     vec4 ndc = vec4(uv * 2.0 - 1.0, depth * 2.0 - 1.0, 1.0);
3     vec4 view = gbufferProjectionInverse * ndc;
4     return view.xyz / view.w;
5 }
```

若还需世界坐标，再左乘视图矩阵的逆 `gbufferModelViewInverse`：

$$\vec{v}_{\text{world}} = V^{-1} \vec{v}_{\text{view, homog}}$$

这一 `screenToView/screenToWorld` 函数对，是本章及第 4 章几乎所有代码的公共基础设施——无论是重建光照所需的视图坐标，还是重建阴影采样所需的世界坐标，都依赖同一套逆变换。

4.2 Phong 光照模型

Phong 模型（1975）将反射光分解为三项：

$$\vec{C} = \vec{C}_{\text{ambient}} + \vec{C}_{\text{diffuse}} + \vec{C}_{\text{specular}}$$

环境光是常数项，粗略近似间接光照：

$$\vec{C}_{\text{ambient}} = k_a \vec{C}_{\text{albedo}}$$

漫反射遵循 Lambert 余弦定律，强度正比于入射光方向在法线上的投影：

$$\vec{C}_{\text{diffuse}} = \vec{C}_{\text{albedo}} \cdot \vec{C}_{\text{light}} \cdot \max(\vec{N} \cdot \vec{L}, 0)$$

镜面反射依赖反射向量 $\vec{R}$ 与视线方向 $\vec{V}$ 的夹角。反射向量的推导：将 $\vec{L}$ 沿法线分解为平行与垂直分量

$$\vec{L} = \vec{L}_{\parallel} + \vec{L}_{\perp}, \quad \vec{L}_{\parallel} = (\vec{L} \cdot \vec{N}) \vec{N}, \quad \vec{L}_{\perp} = \vec{L} - (\vec{L} \cdot \vec{N}) \vec{N}$$

反射时平行分量不变、垂直分量取反：

$$\vec{R} = \vec{L}_{\parallel} - \vec{L}_{\perp} = 2(\vec{L} \cdot \vec{N}) \vec{N} - \vec{L}$$

可验证 $\vec{R} \cdot \vec{N} = \vec{L} \cdot \vec{N}$ 且 $\|\vec{R}\| = \|\vec{L}\|$ ，即入射角等于反射角。镜面项为

$$\vec{C}_{\text{specular}} = k_s \vec{C}_{\text{light}} (\max(\vec{V} \cdot \vec{R}, 0))^n$$

其中 n （`shininess`）越大，高光越集中、越接近镜面。

4.3 Blinn-Phong 改进

Blinn-Phong 用半角向量 $\vec{H}$ ($\vec{L}$ 与 $\vec{V}$ 夹角的角平分线, 由向量加法平行四边形法则得到) 替代反射向量:

$$\vec{H} = \frac{\vec{L} + \vec{V}}{\|\vec{L} + \vec{V}\|}, \quad \vec{C}_{\text{specular}} = k_s \vec{C}_{\text{light}} (\max(\vec{N} \cdot \vec{H}, 0))^n$$

其优势是在 $\vec{V}$ 接近 $-\vec{L}$ (掠射角) 时数值更稳定, 不会出现 Phong 模型中反射向量突变导致的高光断裂。两者存在近似关系

$$\vec{N} \cdot \vec{H} \approx \cos \frac{\theta_{LV}}{2}, \quad \vec{R} \cdot \vec{V} = \cos \theta_{LV}$$

半角把夹角折半, 衰减比 Phong 更平缓, 工程上通常取 Blinn-Phong 的指数为 Phong 的约 4 倍以匹配相近的高光尺寸。

4.4 deferred.fsh 的完整实现范式

Listing 4: deferred.fsh 范式

```

1 uniform sampler2D colortex0;    // albedo
2 uniform sampler2D colortex1;    // 编码法线 (+深度, 视具体打包方案而定)
3 uniform sampler2D depthtex0;
4 uniform mat4 gbufferModelViewInverse;
5 uniform mat4 gbufferProjectionInverse;
6 uniform vec3 shadowLightPosition;
7
8 vec3 screenToView(vec2 uv, float depth) { /* 见 SS3.1 */ }
9
10 void main() {
11     vec4 albedo = texture(colortex0, texcoord);
12     vec3 normal = normalize(texture(colortex1, texcoord).xyz * 2.0 - 1.0);
13     float depth = texture(depthtex0, texcoord).r;
14     vec3 viewPos = screenToView(texcoord, depth);
15
16     vec3 lightDir = normalize(shadowLightPosition);
17     vec3 viewDir = normalize(-viewPos);    //
18     // 相机在原点, 天然指向相机
19
20     float NdotL = max(dot(normal, lightDir), 0.0);
21     vec3 ambient = albedo.rgb * 0.1;
22     vec3 diffuse = albedo.rgb * NdotL * 0.9;
23
24     vec3 halfwayDir = normalize(lightDir + viewDir);
25     float spec = pow(max(dot(normal, halfwayDir), 0.0), 32.0);
26     vec3 specular = vec3(1.0) * spec * 0.3;
27
28     color = vec4(ambient + diffuse + specular, albedo.a);
29 }

```

关于额外光照项的两点推广 (工程实践中常见于此结构之上):

- **背光填充 (Sky Fill):** 以 $(1 - N \cdot L)$ 加权补偿背光面, 近似天空散射光对阴影区的照亮, 避免纯 Lambert 模型下背光面死黑:

$$\vec{C}_{\text{skyfill}} = k_{\text{sky}} \vec{C}_{\text{albedo}} (1 - \max(\vec{N} \cdot \vec{L}, 0))$$

- **距离雾 (指数雾):** 以视距 $d = \|\vec{v}_{\text{view}}\|$ 为自变量, 指数衰减更符合大气散射的物理规律, 且近处过渡比线性雾更自然:

$$f(d) = 1 - e^{-d \cdot \rho}, \quad \vec{C}_{\text{final}} = \text{mix}(\vec{C}_{\text{lit}}, \vec{C}_{\text{fog}}, \text{clamp}(f(d), 0, 1))$$

其中 ρ 为雾浓度系数。

4.5 延迟渲染的性能考量

带宽是延迟渲染的主要瓶颈: 每个 G-Buffer 附件都要写一次、读一次, 附件数越多、精度越高, 带宽压力越大。优化策略包括通道打包 (§2.3)、按需降低非关键附件精度、以及把不必要的计算移出全屏 Pass。对透明物体 (水、玻璃) 而言, 延迟渲染天然不适用——它们必须退回前向渲染单独处理, 这也是 Iris 管线中专设 Translucent 阶段的原因。

5 第 4 章 Shadow Mapping 阴影

阴影是本教程数学密度最高的主题：它把”坐标空间变换”与”深度比较误差”两条线索交织在一起。以下内容按”原理 → 数学形式 → 误差来源 → 修正手段”的顺序展开。

5.1 核心思想

Shadow Mapping 由 Lance Williams 于 1978 年提出，核心命题是：

若光源看不到某点，则该点处于阴影中。

算法分两个 Pass：

- **Shadow Pass:** 把相机搬到光源位置、朝向光照方向，只渲染深度（不渲染颜色），结果写入 `shadowtex0`；
- **Main Pass:** 对每个待着色像素，将其世界坐标变换到光源空间，比较该点在光源空间的深度与 `shadowtex0` 中记录的深度——若前者更远，说明光源与该点之间存在遮挡物，该点在阴影中。

5.2 数学推导

设世界坐标点 $\vec{p}$ 。步骤 1，世界空间 → 光源视图空间：

$$\vec{v}_{\text{lightView}} = V_{\text{shadow}} \cdot \vec{p}$$

步骤 2，光源视图空间 → 光源裁剪空间。太阳/月亮是方向光（平行光），故 P_{shadow} 为正交投影（不同于相机的透视投影）：

$$\vec{v}_{\text{lightClip}} = P_{\text{shadow}} \cdot \vec{v}_{\text{lightView}}$$

步骤 3，裁剪空间 → NDC → 纹理坐标：

$$\vec{s} = \left(\frac{\vec{v}_{\text{lightClip}}}{w_{\text{lightClip}}} \right) \cdot 0.5 + 0.5$$

（正交投影下 $w \equiv 1$ ，此步不产生透视畸变。） $\vec{s}_{xy}$ 用于采样 `shadowtex0`， $\vec{s}_z$ 是该点在光源空间的归一化深度。步骤 4，深度比较，可写成指示函数：

$$\text{shadow}(\vec{p}) = \mathbb{K}[\vec{s}_z \leq \text{shadowtex0}(\vec{s}_{xy}) + \varepsilon_{\text{bias}}]$$

$\mathbb{K}[\cdot]$ 在条件成立时取 1（照亮），否则取 0（遮挡）。这一指示函数是本章后续所有内容的核心对象：偏移量 $\varepsilon_{\text{bias}}$ 的设计 (§4.4) 决定阴影是否失真，而 PCF 软阴影 (§4.5) 本质上是对同一指示函数在空间邻域内取期望。

GLSL 实现：

Listing 5: Shadow Mapping 核心实现

```
1 vec3 screenToWorld(vec2 uv, float depth) {
2     vec4 ndc = vec4(uv * 2.0 - 1.0, depth * 2.0 - 1.0, 1.0);
3     vec4 view = gbufferProjectionInverse * ndc; view /= view.w;
4     return (gbufferModelViewInverse * view).xyz;
5 }
```

```

6
7 float sampleShadow(vec3 worldPos) {
8     vec4 lightView = shadowModelView * vec4(worldPos, 1.0);
9     vec4 lightClip = shadowProjection * lightView;
10    vec3 coord = lightClip.xyz / lightClip.w * 0.5 + 0.5;
11
12    float shadowDepth = texture(shadowtex0, coord.xy).r;
13    float currentDepth = coord.z;
14    float bias = 0.005;
15    return currentDepth - bias > shadowDepth ? 0.0 : 1.0;
16 }

```

5.3 阴影图的空间分辨率

shadowtex0 是固定分辨率纹理(通常 1024^2 或 2048^2),覆盖以玩家为中心、半径为 shadowDistance 的正交投影区域。单个 texel 覆盖的世界空间宽度为

$$w_{\text{texel}} = \frac{2 \cdot D_{\text{shadow}}}{R}$$

例如 $D_{\text{shadow}} = 128$ 、 $R = 2048$ 时, $w_{\text{texel}} = 0.125$ 方块——这是阴影精度的理论上限,远处物体的阴影边缘必然出现锯齿。

5.4 Shadow Acne: 量化误差的机理与修正

现象: 向阳表面出现高频条纹状自阴影,相机移动时随深度采样抖动而闪烁。

机理: 阴影图以有限分辨率离散采样深度场。同一 texel 覆盖的表面区域内,深度比较是硬阈值判定,当相邻像素的真实深度差 Δd 与阴影图的深度量化步长同量级时,比较结果的符号会在微观起伏下随机翻转,形成与阴影图采样频率相同的条纹噪声。

量化分析: 设阴影图覆盖世界宽度对应单 texel 宽度 w ,表面法线与光线夹角为 θ ,则该 texel 在倾斜表面上“展开”的有效长度为

$$\ell_{\text{eff}} = \frac{w}{\cos \theta}$$

由此产生的深度量化误差近似为

$$\varepsilon(\theta) \approx w \tan \theta$$

要让偏移量恰好抵消误差, $\varepsilon_{\text{bias}}$ 必须随倾角增长,这就是斜率比例偏移 (Slope-Scale Bias) 的来源:

$$\varepsilon_{\text{bias}}(\theta) = k \tan \theta, \quad \tan \theta = \frac{\sqrt{1 - (\vec{N} \cdot \vec{L})^2}}{\vec{N} \cdot \vec{L}}$$

三种工程解法,复杂度递增、精度递增:

(1) 恒定偏移——简单但对陡峭表面不足:

```

1 float bias = 0.005;

```

(2) 斜率比例偏移——按法线与光线夹角自适应:

```

1 float bias = max(0.05 * (1.0 - dot(normal, lightDir)), 0.005);

```

(3) 法线偏移——在 Shadow Pass 中直接把几何沿法线方向轻微推移,从源头消除自阴影:

```

1 worldPos += normal * 0.05;

```

5.5 Peter-Panning 效应

偏移量若过大，会导致阴影与物体底部脱节，形似物体悬浮于阴影之上（得名于彼得潘没有影子附着于脚下的意象）。这是 Shadow Acne 修正的代价，而非独立问题——两者构成同一参数 $\varepsilon_{\text{bias}}$ 的两侧权衡：

$$\varepsilon_{\text{bias}} \text{ 过小} \Rightarrow \text{Shadow Acne} \quad \varepsilon_{\text{bias}} \text{ 过大} \Rightarrow \text{Peter-Panning}$$

缓解手段是采用斜率比例偏移（按需分配偏移量，而非全局定值）而非依赖 PCF/PCSS 掩饰硬阈值判定引入的宏观误差。

5.6 PCF 软阴影

基础 Shadow Mapping 输出的是硬阴影——指示函数 $\mathcal{K}[\cdot]$ 只取 0 或 1，而真实阴影存在半影（Penumbra）过渡区。百分比渐近滤波（Percentage Closer Filtering, PCF）的思路是对该指示函数在阴影图空间的邻域内取样本均值，将其转化为连续值：

$$\text{shadow}_{\text{PCF}}(\vec{p}) = \frac{1}{|\Omega|} \sum_{(\Delta x, \Delta y) \in \Omega} \mathcal{K}[\vec{s}_z - \varepsilon_{\text{bias}} \leq \text{shadowtex0}(\vec{s}_{xy} + (\Delta x, \Delta y))]$$

Ω 是采样核（如 3×3 规则网格、或泊松盘随机偏移集合）。 3×3 PCF 实现：

Listing 6: 3×3 PCF 实现

```
1 float sampleShadowPCF(vec3 worldPos, float bias) {
2     vec3 coord = /* 同 SS4.2 步骤 1-3 */;
3     float shadow = 0.0;
4     float texelSize = 1.0 / 2048.0;
5     for (float x = -1.0; x <= 1.0; x += 1.0)
6     for (float y = -1.0; y <= 1.0; y += 1.0) {
7         float d = texture(shadowtex0, coord.xy + vec2(x, y) * texelSize).r;
8         shadow += (coord.z - bias > d) ? 0.0 : 1.0;
9     }
10    return shadow / 9.0;
11 }
```

表 7: PCF 采样核对比

采样核	样本数	特点
2×2	4	速度快，阴影较软，性能敏感场景首选
3×3	9	质量/性能平衡点，多数光影包的标准选择
5×5	25	阴影柔和，开销大，通常需降采样优化
泊松盘	可调	固定半径内随机分布，配合抖动避免网格状条带

硬件比较采样：将 shadowtex0 声明为 sampler2DShadow 而非 sampler2D，texture() 会直接返回硬件执行的深度比较结果，双线性过滤模式下单次采样即等价于 2×2 PCF，但只消耗一次采样指令——这是性能敏感场景的推荐做法。

对于覆盖范围较大的正交阴影图，远处精度不足会造成**阴影失真**，工程上以级联阴影图（CSM，将视锥体分割为多层级、各自独立阴影图）缓解，属于超出 Base-330 范畴的进阶技术。

5.7 最小可用实现

shadow.vsh (结构与普通 G-Buffers 顶点着色器几乎一致):

```

1 out vec2 texcoord;
2 void main() {
3     gl_Position = ftransform();
4     texcoord = (gl_TextureMatrix[0] * gl_MultiTexCoord0).xy;
5 }

```

shadow.fsh (只需 Alpha 测试, 深度由 OpenGL 自动写入, 无需显式操作):

```

1 uniform sampler2D gtexture;
2 uniform float alphaTestRef = 0.1;
3 in vec2 texcoord;
4 /* RENDERTARGETS: 0 */
5 layout(location = 0) out vec4 color;
6
7 void main() {
8     if (texture(gtexture, texcoord).a < alphaTestRef) discard;
9 }

```

deferred.fsh 中接入斜率比例偏移与越界处理:

Listing 7: Deferred 阶段接入阴影

```

1 float sampleShadow(vec3 worldPos, vec3 normal) {
2     vec3 lightDir = normalize(shadowLightPosition);
3     float bias = max(0.05 * (1.0 - dot(normal, lightDir)), 0.005);
4
5     vec4 lightView = shadowModelView * vec4(worldPos, 1.0);
6     vec4 lightClip = shadowProjection * lightView;
7     vec3 coord = lightClip.xyz / lightClip.w * 0.5 + 0.5;
8
9     if (coord.x < 0.0 || coord.x > 1.0 || coord.y < 0.0 ||
10         coord.y > 1.0 || coord.z > 1.0)
11         return 1.0; // 超出阴影图覆盖范围, 视为无遮挡
12
13     float shadowDepth = texture(shadowtex0, coord.xy).r;
14     return coord.z - bias > shadowDepth ? 0.0 : 1.0;
15 }

```

调参启发式: 出现条纹 → 增大 bias; 阴影与物体脱节 → 减小 bias 或改用斜率比例偏移; 阴影边缘生硬 → 引入 PCF。